

THE UNIVERSITY OF DSCHANG
University Institute of Technology Fotso Victor

Department of Computer Engineering

Course material

INFORMATION SYSTEM 1

Narcisse TALLA

*Department of Computer engineering
University Institute of Technology Fotso Victor
University of Dschang
narcisse.talla@univ-dschang.org*

September 2024

TABLE OF CONTENTS

Chapter I. THE INFORMATION SYSTEM	3
1.1. Objective	3
1.2. Definitions	3
1.2. The concept of "information system"	4
1.3. The systemic.....	4
1.4. Some methods of analysis	5
1.4.1. Functional oriented models	5
1.4.1.1. The method FAST (Function Analysis Systems Techniques)	5
1.4.1.2. The Sequential Function Chart (SFC) method or GRAFCET	8
1.4.2. Relational oriented methods	9
1.4.2.1. The SADT method.....	9
1.4.2.2 The method MERISE.....	11
1.4.3 Object Methods	11
1.4.3.1 Object Modelling Technique	11
1.4.3.2 Unified Modelling Language	12
Chapter II. Study of a modelling method : MERISE.....	14
2.1. Introduction.....	14
2.2. Presentation of MERISE	14
2.2.1. Definition and generalities	14
2.2.2. Abstraction cycle of Information System conception	15
2.3. The Conceptual Model of Communication (CMC)	16
2.3.1. Definition of the Organisation	16
2.3.2. The Context Diagram	17
2.3.3. Conceptual Diagram of information flow	18
2.4. The Conceptual Model of Data (CMD)	19
2.4.1. The purpose	19
2.4.2. Entities and class of entities.....	19
2.4.3. Relations and class of relation	20
2.4.4. The cardinality	21
2.4.5. Identifiers.....	22
2.4.6. Aggregation (or relative identification)	23
2.5. The logical model of data (MLD)	23
2.5.1. Description	23
2.5.2. Translation of a class of entity	23
2.5.3. Translation of a class of relation	24
2.5.4. Translation of an aggregation class	24
2.6. The Operating Conceptual Model (OCT)	24
2.6.1. Introduction	24
2.6.2. The concept of event.....	25
2.6.3. Process	25
2.6.7. Operation.....	25
2.6.8. The synchronisation	25

2.6.9. The construction of the operating conceptual model (MCT)	25
2.7. The Operating Organisational Model (MOT)	26
2.7.1. Description	26
2.7.2. The table of functional procedures	26
2.8. The physical model of data	27
2.9. Exercises	27
2.9.1. Management of a library	27
2.9.2. The management of an urban transport company	27
Chapter III. EXAMPLE OF DATA BASE CREATION.....	28
3.1 Problem	28
3.2 Solution	28
3.2.1. Analysis	28
3.2.2. The Logical Model of Data.....	29
3.2.3. Dada Dictionary	29
BIBLIOGRAPHY.....	32

Chapter I. THE INFORMATION SYSTEM

1.1. Objective

This course aims to familiarise students with basic concepts of Information systems. It presents broadly various techniques of Information system design and management. By the end of this course, each student must be able to:

- Define various concepts related to Information system;
- Given a company, a service of a company or an activity of an organisation:
 - Analyse the system and come out with the expression of needs;
 - Build the various conceptual models ;
 - Build the Logical model of Data

1.2. Definitions

❖ An information is the result of a process of formatting and materialization to communicate a fact or a set of facts that improves the knowledge of an audience. An information is not only a data and a data is not always an information. For a data to be an information, a meaning must be assigned to it and it must bring something new to the audience.

❖ A system is a set of dynamically interacting elements organized according to a goal. The interaction includes communication.

❖ An **open system** is a system in permanent interaction with its environment: It is a reservoir that fills up (energy, matter, information) and empties (entropy, energy waste) at the same speed to stay within a given state. It is in constant interaction with its environment, modifying it and in return being modified.

❖ A **closed system** is a system that doesn't exchange matter, information, or energy with its environment: it lives on its internal energy and as feedback, increases its entropy which becomes maximum.

❖ A **complex system** is a system made of a wide variety of components or parts with specialised functions:

- Elements organised in **hierarchical levels**,

- Elements and levels connected by multiple **links**,
- **Non-linear** interactions,
- Difficult and **unpredictable** behaviour
- **Great resistance** to change.

❖ An **information system** is a system that helps decision makers to manage (delete, register, modify, retrieve) information of a company or an organization.

Any system has two aspects:

- ❖ The structural aspect (structural or spatial description).
- ❖ The functional aspect (temporal description).

1.2. The concept of "information system".

The concept of information system is a product of the general systems theory, developed in the 1940s. What characterises the systemic approach can be seen as a reaction to an analytical approach typical of the Cartesian logic and it should be noted that systemic stresses on the concept of totality.

For example, some authors define a system as an organised whole, made of elements that can only be described relatively to each other, according to their place in this totality. Others consider it as a set of units in mutual interrelations, or a set of elements linked by a set of relations.

In this course, we consider a system as a set of entities in interactions, organised to achieve a specific goal, including information management. An entity being a material or immaterial representative of the real world.

The systemic concept distinguishes systems and subsystems. For example in a car system, the engine which is also a system (made of piston, crankshaft, injector, ...) is an entity. It is then a subsystem. In the human body, the digestive system and the nervous system are subsystems.

1.3. The systemic

To examine the infinitely small and the infinitely large, scientists use tools like the microscope and the telescope that have led to significant leaps of knowledge. Today, we are

confronted with the **infinitely complex** and there are no tools to study the tremendous complexity of the systems we are elements and particles. The systemic approach aims to develop a symbolic instrument called **macroscope**, to study this complexity. However, the systemic approach does not pretend to explain everything or to solve every problem and is not intended to provide models of the world that claim to encompass everything.

The natural tendency of the human spirit is unifying, reductive, approximating; It gives apparently satisfactory model, but dangerous: one filters and eliminates all what is different from the unified model and comes out with the worst intransigence. The systemic models are starting points for reflection; they must be confronted with reality, they must be attacked, destroyed, to be better rebuilt because they can **evolve** only in **confrontation**.

1.4. Some methods of analysis

- ❖ In computer science analysis, there are several methods organised in three main categories, notably:
 - Relational oriented models including MERISE, SA, SART SADT;
 - Functional oriented models including FAST, GRAFCET, APTE, RELIASEP;
 - Object oriented models including OMT, UML.

1.4.1. Functional oriented models

1.4.1.1. The method FAST (Function Analysis Systems Techniques)

The method FAST is a technique to develop a graphical representation showing the logical relationships between the functions of a project, product, process or service based on the questions “How” and “Why”. It aids in thinking about the problem objectively and in identifying the scope of the project by showing the logical relationships between functions. The organization of the functions into a function-logic, FAST diagram enables participants to identify of all the required functions. The FAST diagram can be used to verify if, and illustrate how, a proposed solution achieves the needs of the project, and to identify unnecessary, duplicated or missing functions.

The development of a FAST diagram is a creative thought process which supports communication between team members. The development of a FAST diagram helps teams to:

- ❖ Develop a shared understanding of the project;
- ❖ Identify missing functions;
- ❖ Define, simplify and clarify the problem;
- ❖ Organize and understand the relationships between functions;
- ❖ Identify the basic function of the project, process or product;
- ❖ Improve communication and consensus;
- ❖ Stimulate creativity.

Three key questions are addressed in a FAST Diagram:

- ❖ How do you achieve this function?
- ❖ Why do you do this function?
- ❖ When you do this function, what other functions must you do?

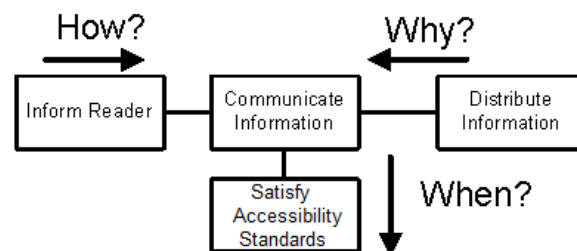


Fig 1. Key components of the FAST method.

The following diagram illustrates how a function is expanded in "How" and "Why" directions in a FAST diagram.

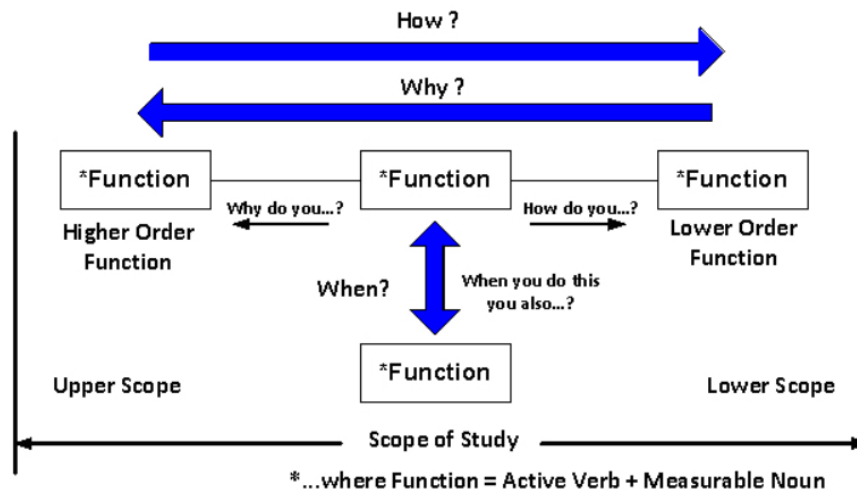


Fig 2. How a function is expanded in "How" and "Why" directions.

Following are the steps in constructing the FAST Diagram. Start with the Functions as identified using Function Analysis:

- ❖ Expand the functions in the "How" and "Why" directions:
- ❖ Build along the "**How**" path by asking 'how is the function achieved'? Place the answer to the right in terms of an active verb and measurable noun;
- ❖ Test the logic in the direction of the "**Why**" path (right to left) by asking 'why is this function undertaken?';
- ❖ When the logic does not work, identify any missing or redundant functions or adjust the order;
- ❖ To identify functions that happen at the same time, ask "when this function is done, what else is done or caused by the function?";
- ❖ The higher order functions (functions towards the left on the FAST Diagram) describe what is being accomplished and lower order functions (functions towards the right on the FAST Diagram) describe how they are being accomplished;
- ❖ "**When**" does not refer to time as measured by a clock, but functions that occur together with or as a result of each other.

1.4.1.2. The Sequential Function Chart (SFC) method or GRAFCET

The **SFC** or **GRAFCET** (**GRA**phe **F**onctionnel de **C**ommande par **E**tapes et **T**ransitions) is a graphical tool that describes the various evolution behaviours of an automated system and establishes a sequential and combinatory correspondance between:

- ❖ **Inputs:** information transfer from Operative section towards Command section;
- ❖ **Outputs:** Information transfer from Command section towards Operative section.

The GRAFCET method was proposed by the "Association Française pour la Cybernétique Economique et Technique" (AFCET). It is a powerful tool, directly exploitable since it is a language used by many existing API. The GRAFCET (capital letters) refers to a modeling tool while a grafcet is a model derived from GRAFCET rules. The GRAFCET comprises:

- ❖ **Steps** associated to actions: A step symbolizes a state or a part of a state of an automated system. To each step is associated one or many actions. An action can be continuous, conditional or memorised;
- ❖ **Transitions** associated to receptivities: a transition indicates the evolution possibility that exists between two steps, i.e. the succession of two activities in the operative section.
- ❖ **Oriented links** among steps and transitions: They are simple vertical lines linking steps to transitions and vice-versa.

The GRAFCET is a graph (diagram) made of alternation of steps and transitions, linked by oriented vertices. It is the method of choice to design structured automation controls. It brings a new dimension to building projects easily and efficiently. Sequential function chart (SFC) / GRAFCET helps design sequences of operation quickly and validates a project prior to its implementation. It complements project documentation by helping engineers, customers, and maintenance personnel to understand equipment operations. Although the interface is simple, there is more depth to this module than meets the eye. During the creation of an Sequential function chart (SFC) / GRAFCET, control elements can be used to implement fault management. Sequence and step execution intervals can be defined. A higher hierarchy level can be exploited by defining macro steps from which entire control structures can be created.

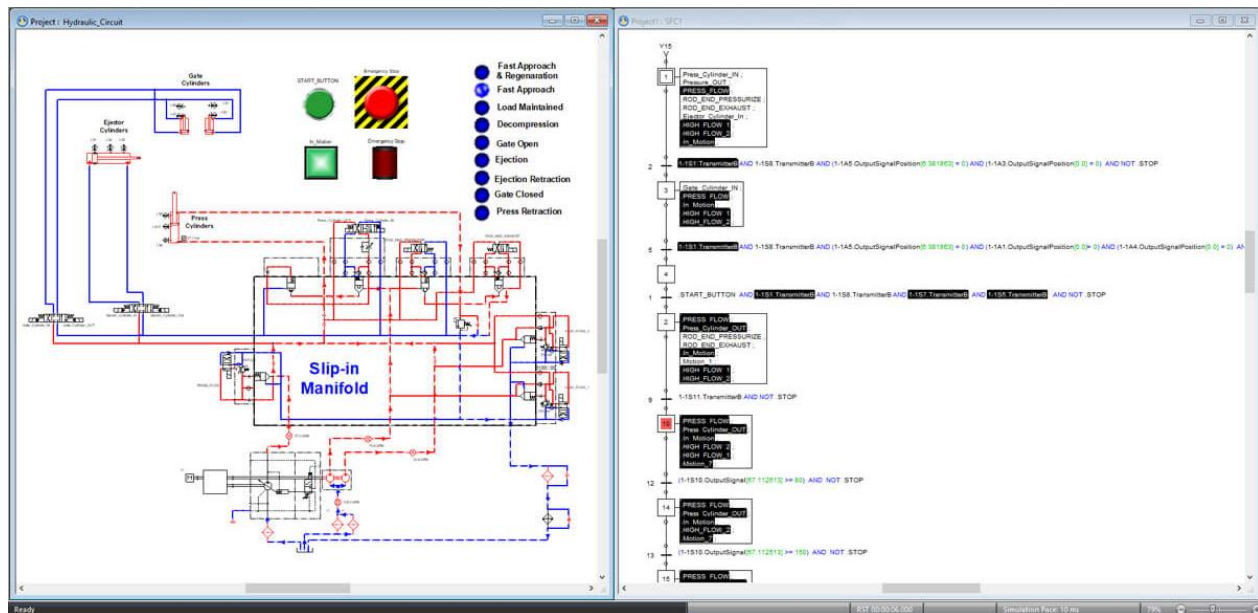


Fig 3. An example of GRAFCET.

1.4.2. Relational oriented methods

1.4.2.1. The SADT method

Definition: The acronym SADT stands for System Analysis and Design Technique. This method has been developed by the Softech Company in the United States. SADT is a method of analysis by successive levels of descriptive approach of a given set. It is then a functional analysis tool of a given process, using successive levels of details.

General principle: The SADT method consists in modelling existing or future systems to understand the functioning and to envisage adequate solutions. This modelling is based on the actions of the analysed system, called actogram, and on the data the system can manage, called datagram, in an arborescent structure of the system.

Originalities: Rather oriented information system, softwares and automatisms, SADT defines two types of stakeholders: the author who conceives the system and the reader who criticises. These works allow to precise and to optimise the system by iterative approach. SADT allows a good traceability and internal controls of the coherence. It can be summarised as:

- A graphical tool of representation;

- ✿ It obliges to record the decisions of the work team. This progressively allows to create a complete documentation of the system.
- ✿ It's a team effort that requires discipline and coordination. Indeed, SADT is a product for communication and diffusion.
- ✿ Its formalism leads to an ascendant or descendant structured representation.
- ✿ If SADT is completely used (actigram and datagram), it allows to program directly an automated system.

Thus, each SADT model consists of a set of activities and a set of objects. Graphical elements and, of course, linguistic labels are used to represent them. The basic building block of SADT is a black box, whose boundary to the environment is defined by the inputs **INPUT** and **CONTROL** and by outputs **OUTPUT** and **MECHANISMS** acting on the element.

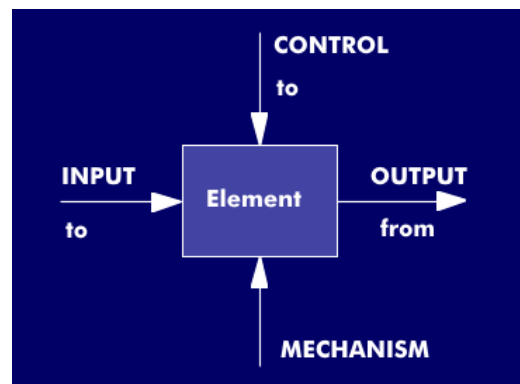


Fig 4. An SADT element.

The SADT method allows to define user needs for IT developments (very used in the industrial Information System) but also to explain and to present an activity's manufacturing processes, procedures. The complete method is heavy and rigorous but allows a very structured, ascending or descending functional analysis. Besides, the SADT is a very accessible graphic representation tool.

SADT's representation is the following:

- ❖ A main box where is specified the name of the process or the action
- ❖ On the left-hand side of this box, incoming arrows: inputs of the action.
- ❖ On the upper part, the incoming arrows: data necessary for the action.
- ❖ On the bottom of the box, incoming arrows: means used for the action.
- ❖ On the right-hand side of the box, outgoing arrows: outputs of the action.

The SADT begins at the level 0 then can be detailed in lower levels (1, 2, 3,...). For example, at the level 1, the level 0 box will be detailed in several elementary boxes and so on...

1.4.2.2 The method MERISE

General principle: The MERISE principle consists in: (1) representing an existing system by easiest invariant elements (sub-systems of order n) which compose the system, (2) optimising links and relations required for the functioning of the system.

Originalities: MERISE is very information system and software-oriented. The next chapter is focus on the MERISE method. The following chapter deeply describes this method.

1.4.3 Object Methods

1.4.3.1 Object Modelling Technique

The object-modelling technique (OMT) is an object modelling approach for software modelling and designing. It was developed as a method to develop object-oriented systems and to support object-oriented programming. OMT describes object model or static structure of the system. It was also developed as an approach to software development. The purposes of modelling are:

- ❖ Testing physical entities before building them (simulation),
- ❖ Communication with customers,
- ❖ Visualization (alternative presentation of information), and
- ❖ Reduction of complexity.

OMT has proposed three main types of models:

- ❖ **Object model:** The object model represents the static and most stable phenomena in the modelled domain. Main concepts are classes and associations with attributes and operations. Aggregation and generalization (with multiple inheritance) are predefined relationships.
- ❖ **Dynamic model:** The dynamic model represents a state/transition view on the model. Main concepts are states, transitions between states, and events to trigger

transitions. Actions can be modelled as occurring within states. Generalization and aggregation (concurrency) are predefined relationships.

- ❖ **Functional model:** The functional model handles the process perspective of the model, corresponding roughly to data flow diagrams. Main concepts are process, data store, data flow, and actors.

OMT is a predecessor of the Unified Modelling Language (UML). Many OMT modelling elements are common to UML.

1.4.3.2 Unified Modelling Language

The Unified Modelling Language (UML) is a general-purpose, developmental, modelling language in the field of software engineering that is intended to provide a standard way to visualize the design of a system. It is originally based on the notations of the Booch method, the object-modelling technique (OMT) and object-oriented software engineering (OOSE), which it has integrated into a single language.

UML is a standardized modelling language consisting of an integrated set of diagrams, developed to help system and software developers for specifying, visualizing, constructing, and documenting the artifacts of software systems, as well as for business modelling and other non-software systems. The UML represents a collection of best engineering practices that have proven successful in the modelling of large and complex systems. The UML is a very important part of developing object oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects. Using the UML helps project teams communicate, explore potential designs, and validate the architectural design of the software. In this article, we will give you detailed ideas about what is UML, the history of UML and a description of each UML diagram type, along with UML examples.

The following figure presents the 14 diagrams of UML.

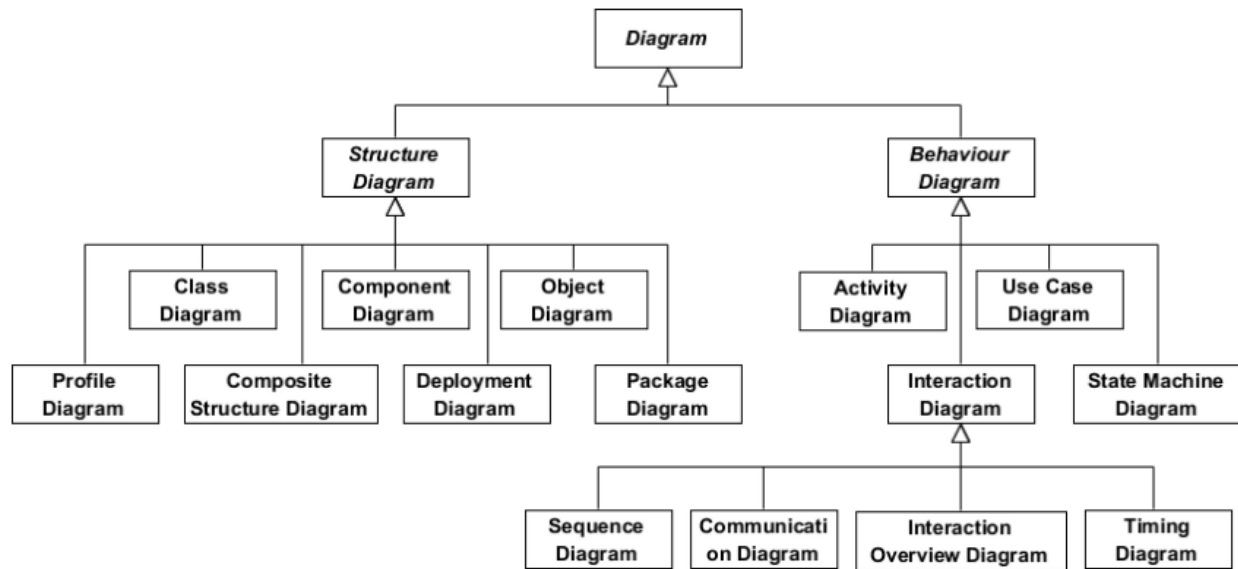


Fig 5. UML diagrams.

Chapter II. Study of a modelling method : MERISE

2.1. Introduction

The conception of an information system is not obvious. Indeed, one must think about the whole organisation to be put in place. The conception phase requires methods allowing building the basic model. The modelling consists of creating a virtual representation of a reality in order to highlight the interesting points. This kind of method is called *analysis*. There are several methods of analysis and MERISE is one of the most used.

2.2. Presentation of MERISE

2.2.1. Definition and generalities

MERISE Stands for “*Méthode pour l’Enseignement et la Recherche en Informatique sur les Systèmes d’Entreprise*”. It is a method of conception, development and realisation of automated systems. The purpose of this method is to conceive an information system. MERISE is based on the separation of data from treatments to be done in several conceptual and physical models called abstraction cycle of MERISE.

The separation of data from operations insures longevity to the model. Indeed, the arrangement of data doesn’t often need to be reshuffled, while operations or treatments are being frequently.

Regarding the company, MERISE distinguishes 3 sub-systems:

- ❖ Steering system;
- ❖ Information System;
- ❖ Operating system.

Where **operating systems** concerns basic operations of the enterprise (production of goods for sale for example) and the **steering system** points to the organisation and decision instances. Between the two, MERISE situates the **information system**.

2.2.2. Abstraction cycle of Information System conception

The conception of information system is done by steps, in order to lead to a functional information system reflecting a physical reality. The purpose is to validate one by one each step, considering the results of the previous steps. Since data are separated from operations, it is required to verify that all data required by the system are present and that there is no superfluous data. This succession of steps is called “abstraction cycle” for the conception of information system:

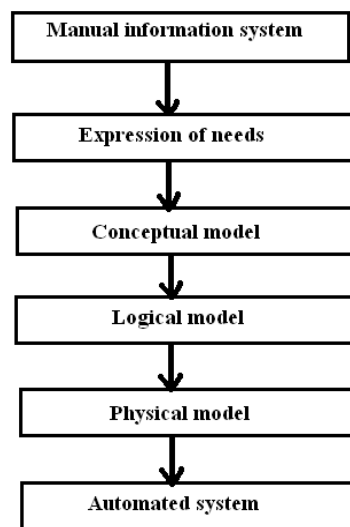


Fig 6. Abstraction cycle of MERISE.

2.2.2.1. The Manual information system

The manual information system is a set of manuscript and oral information used for the management of the structure. This information is generally presented in the form of forms and surveys to fill or in the form of reports to interpret. In the analysis process, this step consists in realising interviews beside different services managers or beside system managers and in collecting samples of each type of information handled in the system. At this level, one must answer to the question: What is done where? How? And by who?

2.2.2.2. The Expression of needs

This step consists in defining what is expected from, and what is required by the automated information system. Therefore, it is required to:

- ❖ Do the inventory of required elements for the information system;
- ❖ Identify the limits or difficulties of the system management;

- ❖ Delimit the system by getting information beside the future users;
- ❖ Define the expected functionalities of the automated system.

2.2.2.3 The Conceptual model

This step consists in realising different conceptual models among which the fundamentals are: the Conceptual Model of Data (CMD), the Operating Conceptual Model (OCM) and the Operating Organisational Model.

The CMD is the model on which are based the data of the system. It describes the architecture of the database and presents the rules and constraints to consider while building the database.

After the expression of needs, the specification of the Conceptual Model of Communication (CMC) which defines the information flows to consider can be done. These information flows start from an actor to another actor of the system and specify the nature or the type and the orientation of the transmitted information. The Operating Organisational Model (OOM) describes the constraints due to the environment (organisational, spatial and temporal).

2.2.2.4 The Logical Model

The Logical Model represents a software choice for the information system. It is deduced from the CMD. It helps in writing queries to access databases. It presents the database in the relational form and expresses the associated functional dependences in the form of relations.

2.2.2.5 The Physical Model

The physical model of an information system reflects the matter choice (for example, Ms Access, Oracle, Paradox, Mysql, etc.) for the information system. It presents the organisation of the created tables and the different relations between them.

2.3. The Conceptual Model of Communication (CMC)

2.3.1. Definition of the Organisation

The first step of this model is to isolate the system by delimiting its domain. The purpose is to define the system and the external elements which exchange information flows with the system. These external elements are called external stakeholders/actors or partners. Information exchanged can be either in only one direction (from external actor to the system

exclusively or from the system to an external actor exclusively) or in double direction (simultaneously from the external actor to the system and vice versa).

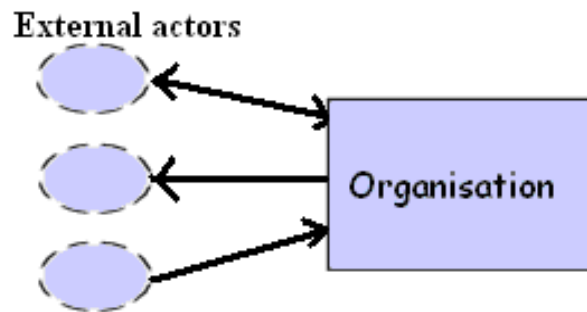


Fig 7. Context diagram formalism.

The second step consists in cutting up the organisation into entities called internal stakeholders/actor (or domaine). When the domain of an organisation is very large, it can be cut up again into sub-domains.

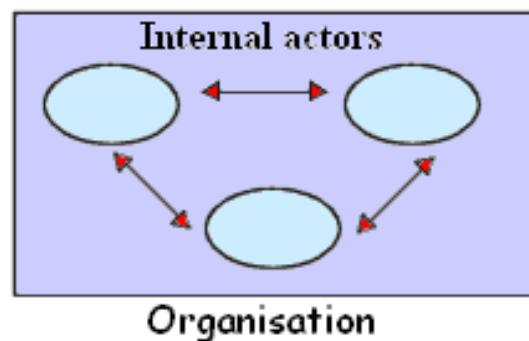


Fig 8. Cutting-up the system domain.

- The last step is the analysis of the information flows, that is, the definition of process.

2.3.2. The Context Diagram

The context diagram aims to represent the information flows between the organisation and the external actors accordingly to a standardised representation in which each object has a name. Notably:

- ❖ The organisation is represented by a rectangle,
- ❖ External actors are represented by ellipses in dotted line,
- ❖ The information flow is represented by an arrow which orientation specifies the direction of the information flow.

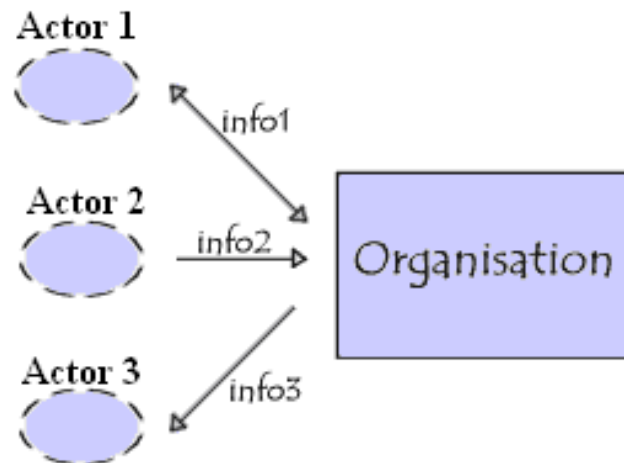


Fig 9. A context diagram of a system.

No internal actor is involved in this diagram. In this diagram, the external environment of the Organisation is made of three actors, namely Actor 1, Actor 2 and Actor 3.

2.3.3. Conceptual Diagram of information flow

This diagram (also called Conceptual Model of Communication) helps to complete the context diagram by breaking down the organisation into a series of internal actors. In this diagram, the standard representation is as follow:

- ❖ Internal actors are represented by ellipses in continuous line;
- ❖ Internal or external messages are represented by oriented arrows;

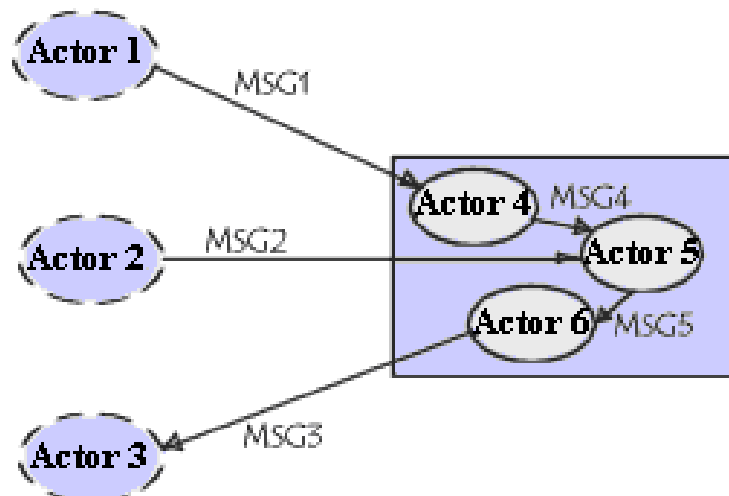


Fig 10. Conceptual Model of Communication formalism.

This diagram can be put in a matrix form called **information flow matrix**. The structure of the said matrix is the following:



	Actor 1	Actor 2		Actor n
Actor 1				
Actor 2				
...				
Actor n				

Each cell localized by the line of an actor J and the column of an actor I specifies the information sent by the actor J to the actor I. If the cells of the matrix are symmetrically filled then all the arrows of the CMC are bidirectional.

The diagonal is black-colored because an actor cannot send an information to himself.

2.4. The Conceptual Model of Data (CMD)

2.4.1. The purpose

The CMD aims to describe formally the data to be used in the information system. It is question here of a data representation, easily comprehensible, allowing describing the information system using entities.

This conceptual model is the main purpose of this course. Each student, by the end of this course, should be able to build the conceptual model of data of any information system.

2.4.2. Entities and class of entities

An **entity** is the representation of a material or immaterial element having a role in the information system that one wishes to describe. It is anything, material or logical that one can manipulate.

A **class of entities** is a set of entities of the same type, (having the same definition). An entity is an instantiation of the class. Each entity is composed of properties, elementary data describing the entity. A class of entities is an abstraction description of properties that all the related entities have in common.

Let us consider the example of a Toyota Advensis, a Mercedes ML and a Renault Megane. These are three entities belonging to the class “Car”. The Toyota Advensis is then an instantiation of the class “Car”. Each entity can possess the properties: colour, year, number of seats and model.

A class of entities is represented by a rectangle separated into two fields as shown in the following figure:

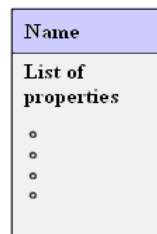


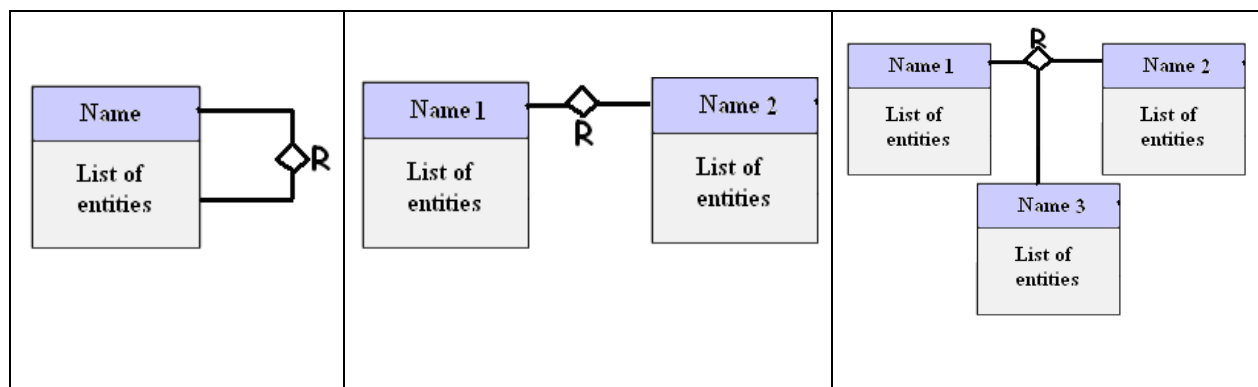
Fig 11. Formalism of a class of entities

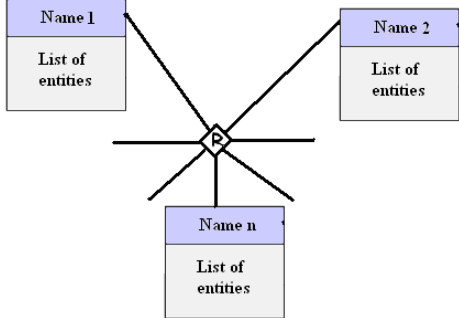
The upper field contains the name of the class. This name is generally an abbreviation for a matter of writing simplification. The question here is to verify that, to each class of entities corresponds one and only one name and vice versa, for a given system.

The lower field contains the list of properties of the class.

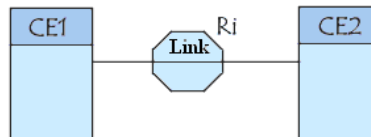
2.4.3. Relations and class of relation

A *relation* (also called *association*) represents the semantic links that can exist between several entities. A *class of relation* contains relations of the same type (It links entities belonging to the same class of entities). A class of relation can link more than two classes of entities. According to the number of participants in a relation, we have the following denominations:



A recursive (reflexive) class of relation links a class of entity to itself.	A binary class of relation links two classes of entity	A ternary class of relation links three classes of entity
		
	An order n class of relation links n classes of entity.	

Classes of relation are represented by *hexagons* (at times by *ellipses*). The name (generally a verb) describes the type of the relation that links the classes. For each class of relation, an identifier R_i is defined. This identifier defines one and only one class of relation.

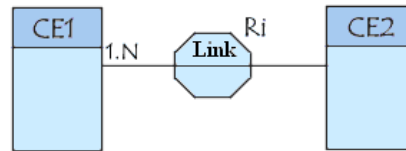


One can eventually add properties to classes of relation. These properties are called *association properties*. These properties cannot belong to any class of relation taking part to the association. For example, the mark is an association property between the class of entity *Course* and the class of entity *student*. The mark is neither a characteristic of the class *Course*, nor a characteristic of the class *Student*, but it exists, due to the fact that a student is in association with a course (a student has written for a course).

2.4.4. The cardinality

Cardinalities characterise the link between an entity and an associated relation. The cardinality of a relation is a couple made up of a minimum limit and a maximum limit defining the interval in which the cardinality of an entity relatively to a given association can take its value:

- ❖ The minimal limit (generally 0 or 1) describes the minimum number of times that an entity of the class of entity can participate to the associated relation.
- ❖ The maximal limit (generally 1 or N) describes the maximum number of times that an entity of the class of entity can participate to the associated relation.



- ❖ A cardinality **1..N** means that each entity belonging to a class of entity participates at least once to the associated relation (link).
- ❖ A cardinality **0..N** means that each entity belonging to a class of entity can participate or not at all to the associated relation (link).
- ❖ A cardinality **0..1** means that each entity of the class of entity participates at most once to the associated relation.
- ❖ A cardinality **1..1** means that each entity of the class of entity participates exactly once to the associated relation.

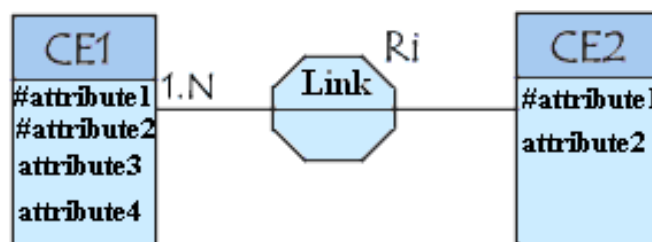
2.4.5. Identifiers

An identifier is a set of properties (one or several) describing one and only one entity. The original definition is the following:

The identifier is a particular property or set of properties of an object so that it's impossible to find two occurrences of the said object for which this (these) property(ies) should have the same value.

Attributes of a class of entity identifying one and only one entity are called *absolute identifiers*.

The MCD formalism proposes to precede identifiers with # or to underline them.



Then, each class of entity must own at least one identifier attribute and the whole set of its identifiers attributes must be field during the creation of the entity. Two different entities

cannot have the value of identifier and the same entity cannot possess two different values of identifier.

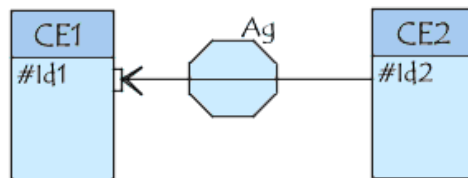
2.4.6. Aggregation (or relative identification)

When an entity identifier is constitute only of intrinsic attributes (attributes that doesn't refer to another entity), it is called *absolute identifier*. Entities made up of absolute identifiers can be defined independently to other entity occurrences. In this case, entities are *independent*.

However, some entities can only be identified through others. This is why we talk of relative identifier. For example, one can talk of the 4th door of the 2nd floor of the building B instead of door N°3451...

Thus, an aggregation specifies that an entity is required for the identification of another one.

- ❖ The class of entity required for the identification of another is called the aggregator class.
- ❖ The class identified is called the aggregated class.



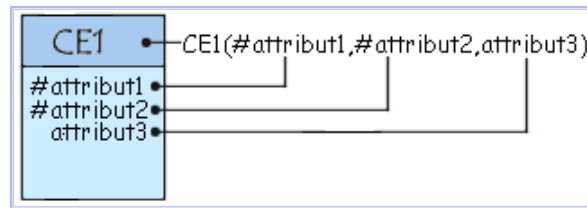
2.5. The logical model of data (MLD)

2.5.1. Description

The logical model of data consists in describing the structure of data used, without however referring to a particular programming language. The purpose is then to precise the type of data used during the process. Thus the logical model depends to the type of data base used.

2.5.2. Translation of a class of entity

Each class of entity of the conceptual model becomes a table in the logical model. The identifiers of the class of entity are called keys of the table while the standard properties become attributes of the table. Each attribute defines a column in the table.

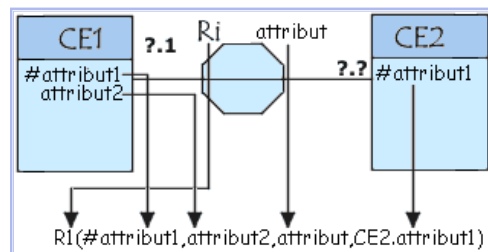


2.5.3. Translation of a class of relation

The passage from the conceptual model to the logical model related to the classes of relation is done accordingly to the cardinalities of each entity associated to the association.

If a class of entity possesses a weak cardinality, the table should have as attributes the properties of the class with weak cardinality, the properties of the relation and finally the primary key of the other class.

If the two classes of entity possess high cardinalities, the table should have as attributes, the primary keys of the two classes and the properties of the association.



2.5.4. Translation of an aggregation class

In the case of an aggregation class, the aggregated entity class has as additional attribute, the primary key of the aggregator class.

2.6. The Operating Conceptual Model (OCT)

2.6.1. Introduction

The operating conceptual model allows studying the dynamic of the information system.

This model allows representing graphically the activity of the information system without considering organisational choices or execution means. It defines simply what is supposed to be done without specifying neither the date, nor how, nor the place.

2.6.2. The concept of event

An event represents a change in the external universe of the Information system or in the information system itself.

- ✚ An external event is a change of the external universe of the information system
- ✚ An internal event is a change in the information system

Events are represented by ellipses in continuous line for internal event and dotted line for external event.



2.6.3. Process

A process is a sub-set of the enterprise activity. Indeed, the activity of the enterprise is made up of a set of process. A process itself is made up of *operations*.

2.6.7. Operation

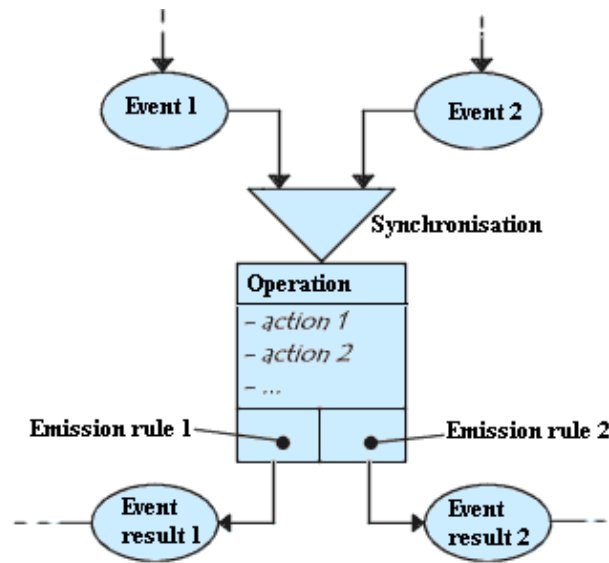
An operation is a set of actions proceeded by the system after an event or a conjunction of events. This set of actions is uninterruptible until the operation is accomplished.

2.6.8. The synchronisation

The synchronisation of an operation defines a Boolean condition on events that contribute to trigger the operation.

2.6.9. The conceptual model of Treatments (CMT)

The conceptual model of treatments allows representing graphically the management of events.



2.7. Organisational Model of Treatments (OMT)

2.7.1. Description

The operating organisational model describes the properties of operations not considered by the conceptual model of data, notably the time, resources and the place.

The OMT consists in representing the CMT in a table which specifies in its columns the duration, the place, the personal chair and the resources required by an action.

2.7.2. The table of functional procedures

The first step of the OMT consists in cutting up operations into functional procedures (FP), a succession of operations triggered by an event. The purpose is to associate in a table:

- ✚ The functional procedures;
- ✚ The beginning and the ending time;
- ✚ The place of the workstation;
- ✚ The personal chair of the workstation;
- ✚ The resources of the workstation.

Procedure	Time		Workstation		
	Begin	End	Place	Personal chair	Resources

2.8. The physical model of data

This step consists in implementing the model in a specific System of data Base Management. This means to translate into a specific definition language. The commonly used language for this type of operation is the SQL (Structured Query Language).

2.9. Exercises

2.9.1. Management of a library

You have to create a data base for the management of a library. The librarian asks you to be able to manage the following information: Books characterised each by a title, the authors (name and surname), an editor, an edition date and the kinds (for example: love, fiction science, politic, roman, news,...) and the customers (name and surname) each with an address (street, postal code and town), the books borrowed, the expected date for book return and the effective date of book return. The state of the book is registered during the borrowing process and verified during the returning process. In case the book is damaged, a penalty is applied.

You don't know more and you are asked to do reasonable hypothesis and choices required by the system. Conceive then the data base.

2.9.2. The management of an urban transport company

You must create a data base for the management of a bus company. The company manager gives you the following indications: each bus is numbered. It realises ways made of a departure, several stops and a terminus. Each bus can every day be allocated to several ways and it can do many journeys on each way with eventually distinct drivers. The management of customers is done separately (in another data base not concerned for this project) but the management of buses (date of the next revision, the mileage, number and name of the depot,...) and drivers (name, surname, grade, seniority, salary, buses droved, ways done,...) is included in this data base.

Chapter III. EXAMPLE OF DATA BASE CREATION

3.1 Problem

3.1.1 Specification

You have to create a data base for the management of a library. The librarian asks you to be able to manage the following information: Books characterised each by a title, the authors (name and surname), an editor, an edition date and the kinds (for example: love, fiction science, politic, roman, news,...) and the customers (name and surname) each with an address (street, postal code and town), the books borrowed, the expected date for book return and the effective date of book return. The state of the book is registered during the borrowing process and verified during the returning process. In case the book is damaged, a penalty is applied.

You don't know more and you are asked to do reasonable hypothesis and choices required by the system. Conceive then the data base.

3.1.2 The expected result.

The data base should produce answers to the following preoccupations:

3.2 Solution

3.2.1. Analysis

One can denote that there are four entities, notably: Book, Customer, Author and Editor.

The related conceptual model is the following:

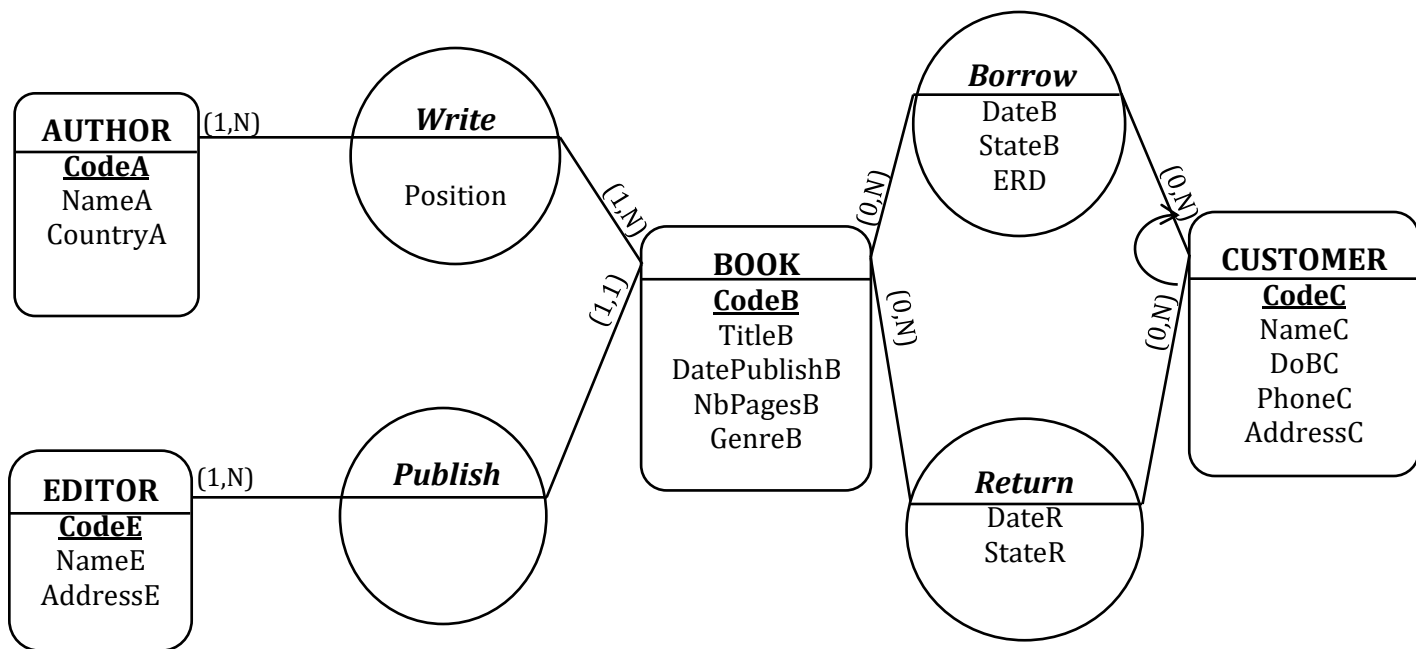


Fig 12. The conceptual model of data (MCD)

3.2.2. The Logical Model of Data

The equivalent **logical model** of data is the following:

Author(CodeA, NameA, CountryA)
 Book(CodeB, TitleB, DatePublishB, NbPagesB, GenreB, EditorB*)
 Editor (CodeE, NameE, AddressE)
 Customer(CodeC, NameC, DoBC, PhoneC, AddressC)
 Return (IdR, BId*, DateR, StateR)
 Borrow (IdB, DateB, StateB, ERD)
 Write(IdW, AutorW, BookW, Position)

3.2.3. Dada Dictionary

Cod	Designation	Type	Lenght	Nature	Reference
CodeA	Identifier cod of the author	AN	6	PK	Author
NameA	Name of the Author	AN	100	AT	Author

CountryA	Country of origin of the Author	A	50	AT	Author
CodeBook	Identifier cod of the Book	AN	8	PK	Book
TitleB	Title of the Book	AN	100	AT	Book
DatePublishB	Publication date of the Book	Date	-	AT	Book
NbPagesB	Number of pages of the Book	N	-	AT	Book
GenreB, EditorB	Literature Genre of the book	A	25	AT	Book
CodeC	Identifier cod of the Customer	AN	8	PK	Customer
NameC	Name of the Customer	AN	100	AT	Customer
DoBC	Date of Birth of the Customer	Date	-	AR	Customer
AddressC	Address of the Customer	AN	200	AT	Customer
PhoneC	Phone Number of the Customer	AN	15	AT	Customer
IdR	Identifier of the Return of a Book	N	-	PK	Return
BId	Borrowing Identifier of the returned Book	N	-	FK	Return
DateR	Date of return of the Book	Date	-	AT	Return
StateR	State of the Book at the last return	A	100	AT	Return
CodeE	Identifier cod of the Editor	AN	8	PK	Editor
NameE	Name of the Editor	AN	100	AT	Editor
AddressE	Address of the Editor	AN	200	AT	Editor
<u>IdB</u>	Borrowing Identifier	N	-	PK	Borrow
DateB	Borrowing Date of the Book	Date	-	AT	Borrow

StateB	State of the Book at the last Borrowing	A	100	AT	Borrow
ERD	Expected returning date of the Book	Date	-	AT	Borrow
<u>IdW</u>	Identifier of the writing of a Book	N	-	PK	Write
AutorW	Identifier cod of the Author who wrote the Book	AN	8	FK	Write
BookW	Identifier cod of the Book written by the author	AN	8	FK	Write
Position	Position of the Author among other authors of the Book	N	-	AT	Write

BIBLIOGRAPHY

[1] Jean-Claude. COURBON, 1997, Dunod, "Systèmes d'information : structuration, modélisation et communication", 288 pages, ISBN : 2729604227.

[2] André MULLER, 2005, Presses Universitaires de France (PUF), "L'informatique dans l'entreprise", 127 pages, 2^{ème} édition, ASIN : 2130445926.

[3] Steven ALTER, 2001, Prentice Hall, "Information Systems: Foundation of E-Business", 587 pages 4^{ème} édition, ISBN: 0130617733.